

HTTP Credential Extractor

PCAP Analysis & Credential Extraction Demo

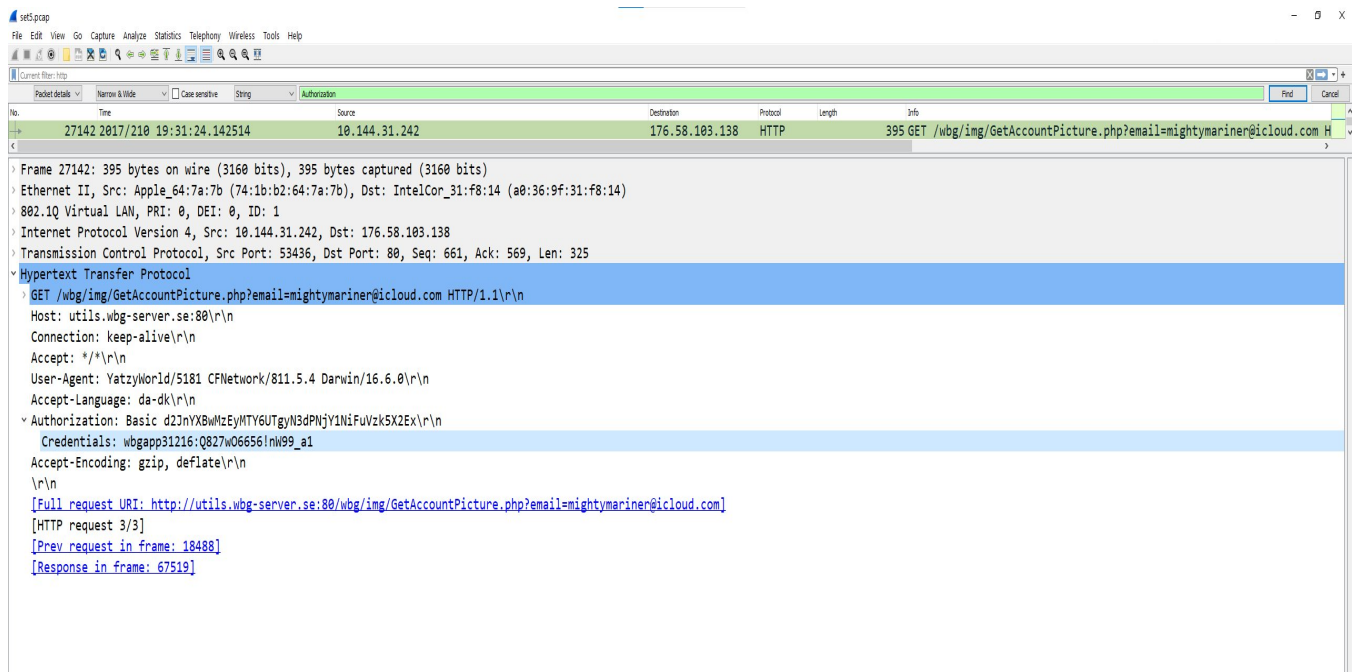
Generated: 2026-06-07 18:49:10

Overview

This report demonstrates the HTTP Credential Extractor tool in action. The tool parses PCAP (Packet Capture) files using Scapy and identifies HTTP requests containing authentication credentials in the Authorization header. It decodes base64-encoded Basic Authentication tokens to reveal plaintext username and password.

Raw HTTP Packet (Wireshark Capture)

Wireshark display of the HTTP packet containing leaked credentials:



The Wireshark capture shows the HTTP GET request to /wbg/img/GetAccountPicture.php with Authorization header containing base64-encoded credentials:
d23nYXBwMzEyMTY6UTgyN3cwNjY1NmFuVzksX2Ix

Tool Execution & Results

Terminal output from running the HTTP credential extractor:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Fehzan\Desktop\python\projects\pcap> python .\http_credential_extractor.py
[+] ALERT. Leaked Credentials Found! -> User: wbgapp31216 | Pass: Q827w06656!nW99_a1
```

Credential Extraction Analysis

Base64 Decoding:

Authorization header value: Basic d23nYXBwMzEyMTZpZXY6UTgyN3cwNjY1NmLuVzk5X2Ix

When decoded: **wbgapp31216:Q827w06656!nW99_a1**

Extracted Credentials:

Username: **wbgapp31216**

Password: **Q827w06656!nW99_a1**

Key Technical Learnings

PCAP Binary Format

PCAP files are binary and cannot be read as text. Scapy's rdpcap() function properly parses the binary format.

Packet Layer Structure

Network packets are nested in layers. Raw payload data must be extracted using syntax like every_packet['Raw'].load

Regex on Bytes

Pattern matching in network payloads requires regex on bytes: re.search(b'pattern', payload)

Multiple Patterns

Using b'PATTERN1|PATTERN2' syntax enables efficient matching of multiple keywords in one regex call.

UTF-8 Decoding

.decode('utf-8', errors='ignore') safely converts binary data to readable strings, ignoring invalid UTF-8 sequences.

Protocol Detection

Port-based detection is unreliable. Protocol-based detection (matching actual protocol headers) is more robust.

Basic Auth Decoding

HTTP Basic auth uses base64-encoded credentials in the format username:password. Decoding the Authorization header and splitting on ':' extracts the actual credentials.

Security Impact

HTTP Basic Authentication is inherently insecure. Credentials are base64-encoded (not encrypted) and visible in plaintext within the HTTP header. This demo illustrates why HTTPS/TLS is essential for protecting authentication credentials in transit. The base64 encoding provides no security benefit—it's merely a transport encoding, and any attacker with network access to unencrypted traffic can easily decode these credentials.